Guía de Estudio para la Exposición

Plataforma de Comercio de Energía — 12 Patrones de Diseño (Backend y Frontend)

> **Propósito de este documento.** Reúne, en un solo lugar, todo lo necesario para estudiar y exponer el proyecto: contexto, arquitectura, los 12 patrones de diseño (cada uno en backend PHP y frontend React/TypeScript), guion sugerido de exposición, demo paso a paso, comparaciones típicas, preguntas frecuentes del jurado y mapeo con la rúbrica. Está pensado para leerse en voz alta o resumirse con notas al margen.

Índice

1. Cómo usar esta guía
2. Resumen ejecutivo del proyecto
3. Arquitectura general
4. Guion de exposición (12–15 minutos)
5. Los 12 patrones de diseño - 5.1 Singleton - 5.2 Factory Method - 5.3 Abstract Method / Template Method - 5.4 Builder - 5.5 Prototype - 5.6 Adapter - 5.7 Bridge - 5.8 Decorator - 5.9 Composite - 5.10 Facade - 5.11 Flyweight - 5.12 Proxy
6. Demo en vivo paso a paso
7. Comparaciones que suelen preguntar
8. Preguntas frecuentes del jurado y respuestas modelo
9. Mapa de la rúbrica y los objetivos del proyecto
10. Endpoints y comandos útiles
11. Glosario rápido
12. Checklist final antes de exponer

---

1. Cómo usar esta guía

Plan sugerido de estudio (3 sesiones):

Sesión	Duración	Qué estudiar	Objetivo
1	60 min	Secciones 1–3 (contexto y arquitectura)	Entender qué es el proyecto y cómo está organizado
2	90 min	Sección 5 (patrones 1–6)	Dominar la primera mitad de patrones
3	90 min	Sección 5 (patrones 7–12) + 6 y 7	Dominar la segunda mitad y la demo
Repaso	30 min	Secciones 8, 9, 12	Anticipar preguntas y verificar checklist

Cómo leer cada patrón (sección 5). Cada patrón tiene siempre la misma estructura:

- **Idea en una frase** → para decir en voz alta.
- **Problema y solución en el proyecto** → el porqué.
- **Participantes y archivos** → el qué (con rutas exactas).
- **Cómo funciona** → el cómo, paso a paso.
- **Código clave** → para señalar en pantalla.
- **Dónde se ve en la app** → la evidencia funcional.
- **Frase de cierre / transición** → para fluir hacia el siguiente patrón.

> **Consejo.** Si el tiempo aprieta, memoriza las ideas en una frase y las frases de transición: son el esqueleto de la exposición. El detalle técnico lo puedes apoyar señalando el código y el README.

---

2. Resumen ejecutivo del proyecto

¿Qué es? Una **Plataforma de Comercio de Energía**: un prototipo web donde los usuarios intercambian **excedentes de energía renovable**. Un usuario que produce más de lo que consume publica su excedente; otro que necesita energía lo compra.

Problema que resuelve. Los excedentes de energía de pequeños productores suelen perderse porque no hay un canal simple para publicarlos, compararlos y venderlos. La plataforma crea un **mercado virtual** con ofertas, compras, subastas, dispositivos IoT simulados y predicciones.

Alcance. Es un **prototipo funcional académico**: no hay pagos reales, ni conexión con redes eléctricas, ni facturación real, ni hardware IoT físico. Todo se simula con un **saldo/transacción virtual** y una **base de datos en memoria persistida en JSON**.

Usuarios:

- **Productor:** genera energía y vende excedentes.
- **Consumidor:** compra energía disponible.
- **Doble rol:** un mismo usuario puede producir y consumir.

Los 4 subsistemas:

1. **Compra y venta de excedentes** — ofertas (kWh y precio), compras e historial.
2. **Subastas en tiempo real** — ofertas con precio inicial y cierre por mejor oferta.

3. **Integración con dispositivos IoT** — medidores/inversores simulados que reportan lecturas.
4. **Predicción de producción y consumo** — estimaciones diaria/semanal y recomendaciones.

Objetivo principal. Desarrollar una plataforma web para gestionar el intercambio de excedentes integrando producción, consumo, excedentes, predicciones, compra, venta e historial, con una **arquitectura organizada, escalable y mantenible**.

Stack tecnológico:

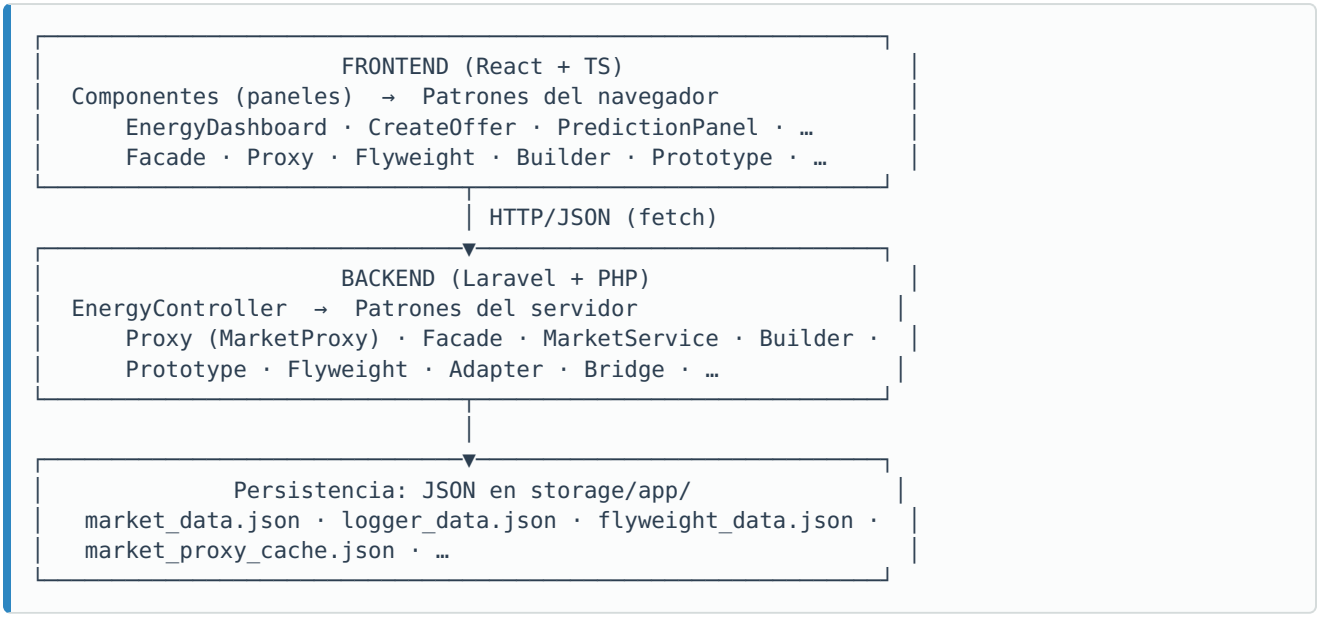
Capa	Tecnología
Frontend	React + TypeScript + Vite
Backend	Laravel + PHP 8
Persistencia	Archivos JSON (`storage/app/*.json`) — base de datos en memoria
Patrones	12 patrones GoF implementados en backend y frontend

Cifras para mencionar: 12 patrones × 2 capas = **~26 implementaciones** documentadas; más de **50 endpoints** REST; **10 paneles** funcionales en la interfaz.

---

3. Arquitectura general

3.1 Vista de capas



3.2 Mapa de carpetas (lo importante)

Backend (`backend-laravel/`):

```
app/Http/Controllers/EnergyController.php     ← endpoints (cliente de los patrones)
app/Models/EnergyOffer.php                    ← oferta de energía
app/Patterns/
├── MarketService.php            (Singleton)  ├── LoggerService.php (Singleton)
├── ConfigService.php            (Singleton)
├── Factories/                   (Factory Method)  EnergySourceFactory, PredictorFactory
├── Predictors/                  (Abstract Method) AbstractPredictor, Daily/WeeklyPredictor
├── Builders/                    (Builder)    EnergyOfferBuilder, OfferDirector
├── Prototypes/                  (Prototype)  Prototype, OfferPrototype, PrototypeRegistry
├── Adapters/                    (Adapter)    DeviceGateway, LegacyScadaAdapter,
PvInverterAdapter
├── Bridges/                     (Bridge)     EnergyBill, CurrencyBridge, SettlementService
├── Decorators/                  (Decorator)  OfferComponent, BasicOffer, 3 decoradores
├── Composite/                   (Composite)  BundleComponent, OfferLeaf, EnergyBundle
├── Facades/                     (Facade)     EnergyPlatformFacade
├── Flyweights/     (NUEVO)      (Flyweight)  EnergySourceFlyweightFactory
└── Proxies/        (NUEVO)      (Proxy)      MarketProxy
routes/api.php                                  ← rutas /api/energy/*
```

Frontend (`frontend/src/`):

```
App.tsx                            ← estructura visual (hero + 10 secciones)
components/                        ← paneles visibles al usuario
│  EnergyDashboard.tsx  CreateOffer.tsx  PredictionPanel.tsx  OfferPrototypePanel.tsx
│  DevicePanel.tsx  SettlementPanel.tsx  OfferExtrasPanel.tsx  BundlePanel.tsx
│  PlatformPanel.tsx  PatternsPanel.tsx (NUEVO)
patterns/
├── singleton/energy-api.service.ts (Singleton)
├── factory/                       (Factory Method)  EnergyCardFactory + cards
├── template/                      (Abstract Method) Predictor, Daily/WeeklyPredictor
├── builder/                       (Builder)         EnergyOfferBuilder, OfferDirector
├── prototype/                     (Prototype)       Prototype, OfferPrototype,
PrototypeRegistry
├── adapter/                       (Adapter)         DeviceGateway, adaptadores, registry
├── bridge/                        (Bridge)          EnergyBill, CurrencyBridge,
SettlementService
├── decorator/                     (Decorator)       OfferComponent, BasicOffer, decoradores
├── composite/                     (Composite)       BundleComponent, OfferLeaf,
EnergyBundle
├── facade/                        (Facade)          PlatformFacade
├── flyweight/  (NUEVO)            (Flyweight)       EnergyTypeFlyweightFactory
└── proxy/      (NUEVO)            (Proxy)           EnergyApiProxy
```

3.3 Flujo típico de una compra (para explicar la integración)

1. El usuario ve **ofertas** en `EnergyDashboard`. El frontend las pide a través del `EnergyApiProxy` (Proxy), que puede servirlas desde su **caché** o delegar en `EnergyApiService` (Singleton).
2. La petición llega a `GET /api/energy/offers`. En el backend, `EnergyController` consulta el `MarketProxy` (Proxy: caché + protección), que delega en `MarketService` (Singleton) si la caché expiró.
3. Cada oferta se **enriquece con los descriptores compartidos** del `EnergySourceFlyweightFactory` (Flyweight): el nombre, color y eficiencia del tipo de energía se guardan una sola vez.

4. Al **comprar**, `POST /api/energy/offers/{id}/purchase` pasa por el **Proxy de protección** (mínimo 1 kWh, máximo 100 kWh) y, si es válido, el mercado actualiza el stock y **invalida la caché**.
5. El **Facade** (`EnergyPlatformFacade` / `PlatformFacade`) permite hacer todo lo anterior en operaciones de alto nivel (panel unificado, publicar, comprar + liquidar, asesoría).

3.4 Tabla maestra de patrones

#	Patrón	Problema que resuelve	Artefactos clave
1	**Singleton**	Una sola instancia/estado compartido	`MarketService`, `LoggerService`, `ConfigService`, `EnergyApiService`
2	**Factory Method**	Crear el objeto correcto según un tipo sin `new` disperso	`EnergySourceFactory`, `PredictorFactory`, `EnergyCardFactory`
3	**Abstract Method** (Template Method)	Reutilizar el esqueleto de un algoritmo variando los datos	`AbstractPredictor` + `Daily/WeeklyPredictor`, `Predictor` + variantes
4	**Builder**	Construir ofertas complejas paso a paso y con variantes	`EnergyOfferBuilder`, `OfferDirector`
5	**Prototype**	Copiar ofertas/plantillas sin reconstruirlas desde cero	`Prototype`, `OfferPrototype`, `PrototypeRegistry`
6	**Adapter**	Unificar dispositivos heredados heterogéneos	`DeviceGateway`, `LegacyScadaAdapter`, `PvInverterAdapter`
7	**Bridge**	Combinar perfiles de facturación × monedas sin explotar clases	`EnergyBill`, `CurrencyBridge`, `SettlementService`
8	**Decorator**	Añadir extras a una oferta dinámicamente	`BasicOffer`, `OfferDecorator`, `OfferPricingService`
9	**Composite**	Tratar ofertas y paquetes de ofertas por igual (recursivo)	`BundleComponent`, `OfferLeaf`, `EnergyBundle`
10	**Facade**	Simplificar el acceso a muchos subsistemas	`EnergyPlatformFacade`, `PlatformFacade`
11	**Flyweight**	Compartir estado común entre muchísimas ofertas	`EnergySourceFlyweightFactory`, `EnergyTypeFlyweightFactory`
12	**Proxy**	Controlar el acceso: caché, virtualización y protección	`MarketProxy`, `EnergyApiProxy`

---

4. Guion de exposición (12–15 minutos)

> Este guion es una **sugerencia**. Los tiempos son orientativos para una exposición de ~14 minutos con demo en vivo.

Tiempo	Bloque	Qué decir (resumen)
0:00–1:00	**Contexto**	Qué es la plataforma, problema, usuarios y los 4 subsistemas.
1:00–2:00	**Arquitectura**	Dos capas (React/TS y Laravel/PHP), persistencia JSON y el flujo de compra.
2:00–3:00	**Singleton**	Un solo mercado, un solo logger, una sola config; un solo cliente HTTP.
3:00–4:00	**Factory Method**	El tipo de energía (`solar/wind/hydro`) decide qué clase y qué tarjeta se crean.
4:00–5:00	**Abstract Method**	Predicción diaria/semanal comparte el esqueleto `predict()` .
5:00–6:00	**Builder**	El asistente de 6 pasos construye la oferta; variante exprés con precio por defecto.
6:00–7:00	**Prototype**	Duplicar una oferta o partir de una plantilla con un “copy”.
7:00–8:00	**Adapter**	Medidores SCADA e inversores PV se ven igual tras la interfaz `DeviceGateway` .
8:00–9:00	**Bridge**	Perfil (residencial/comercial) × moneda (COP/USD/EUR) sin combinatoria de clases.
9:00–10:00	**Decorator**	Extras de oferta (verde +5%, seguro +3%, prioridad $8.000) que se envuelven.
10:00–11:00	**Composite**	Paquetes de ofertas con totales recursivos.
11:00–12:00	**Facade**	Una sola llamada trae mercado, IoT, config y actividad; y publica/compra.
12:00–12:40	**Flyweight**	3 descriptores compartidos para todas las ofertas.
12:40–13:30	**Proxy**	Caché con TTL, inicialización diferida y protección de compras.
13:30–14:00	**Cierre**	Cumplimiento de objetivos, rúbrica y agradecimiento.

Frases de transición que puedes usar tal cual:

- Singleton → Factory: “Ya que todos comparten los mismos servicios, el siguiente reto es **crear el objeto correcto** según el tipo de energía; para eso usamos Factory Method.”
- Factory → Abstract Method: “La fábrica decide **qué** objeto crear; ahora necesitamos que **compartan un algoritmo**; eso lo resuelve el Template Method.”
- Abstract Method → Builder: “Si podemos predecir, también podemos **construir ofertas** paso a paso; ahí entra el Builder.”
- Builder → Prototype: “Construir está bien, pero a veces conviene **copiar**; para eso usamos Prototype.”
- Prototype → Adapter: “Del mercado pasamos al **mundo físico simulado**: sensores y dispositivos que hablan distinto; el Adapter los unifica.”
- Adapter → Bridge: “Con los datos unificados, debemos **facturarlos** en distintos perfiles y monedas; el Bridge separa esas dos dimensiones.”

- Bridge → Decorator: “La factura es el precio base; para **añadir extras** sin tocar la base, usamos el Decorator.”
- Decorator → Composite: “Con las ofertas ya decoradas, podemos **agruparlas en paquetes**; el Composite permite tratarlas todas igual.”
- Composite → Facade: “Ya tenemos muchos subsistemas; para no obligar al cliente a conocerlos, ponemos un Facade.”
- Facade → Flyweight: “El Facade coordina datos; pero con muchas ofertas repetiríamos información. El Flyweight **la comparte**.”
- Flyweight → Proxy: “Y para no consultar el mercado en cada petición y **proteger las compras**, cerramos con el Proxy.”

---

5. Los 12 patrones de diseño

> En cada patrón: primero la idea, luego el detalle. Las rutas de archivo son las reales del repositorio.

5.1 Singleton

Idea en una frase.

> “Garantiza **una única instancia** y un punto de acceso global, para que todos los módulos compartan el mismo estado.”

Problema y solución en el proyecto. El mercado de energía, los logs y la configuración no pueden vivir duplicados: si cada controlador creara su propio `MarketService`, unos verían ofertas que otros no. El Singleton garantiza **un solo mercado, un solo logger y una sola configuración**. En el frontend, un solo cliente HTTP comparte la URL base y evita abrir conexiones innecesarias.

Participantes y archivos (4 implementaciones).

Implementación	Archivo	Rol
`MarketService`	`backend-laravel/app/Patterns/MarketService.php`	Mercado: ofertas + transacciones, persistido en JSON
`LoggerService`	`backend-laravel/app/Patterns/LoggerService.php`	Logs y métricas de actividad
`ConfigService`	`backend-laravel/app/Patterns/ConfigService.php`	Constantes globales (precio, moneda, límites, TTL)
`EnergyApiService`	`frontend/src/patterns/singleton/energy-api.service.ts`	Cliente HTTP único del frontend

Cómo funciona (los 3 pasos clásicos).

1. **Atributo estático privado** que guarda la instancia (`private static ?MarketService $instance = null;`).
2. **Constructor privado** (`private function __construct() {}`) → nadie puede hacer `new` .
3. **Método estático** `getInstance()` que crea la instancia solo la primera vez y luego devuelve siempre la misma.

Código clave (backend):

```
class MarketService
{
    private static ?MarketService $instance = null;
    private array $offers = [];

    private function __construct()      // 1) constructor privado
    {
        $this->dataFile = storage_path('app/market_data.json');
        $this->load();                  // 2) estado persistido, único
    }

    public static function getInstance(): self   // 3) acceso global
    {
        if (self::$instance === null) {
            self::$instance = new self();
        }
        return self::$instance;
    }
}
```

Código clave (frontend):

```
class EnergyApiService {
  private static instance: EnergyApiService;
  private baseUrl = 'http://localhost:8000/api';

  private constructor() {}                          // constructor privado

  static getInstance(): EnergyApiService {         // acceso global
    if (!EnergyApiService.instance) {
      EnergyApiService.instance = new EnergyApiService();
    }
    return EnergyApiService.instance;
  }
}
```

Dónde se ve en la app y datos útiles. `ConfigService` alimenta los límites del mercado: `defaultPricePerKwh = 500` , `currency = COP` , `maxOfferKwh = 1000` , `minPurchaseKwh = 1` , `maxPurchaseKwh = 100` , `offersCacheTtl = 10` segundos. Cada panel del frontend usa el mismo `EnergyApiService.getInstance()` , por lo que todos apuntan al mismo backend.

Frase de cierre / transición.

> “Con un estado único garantizado, el siguiente problema es: **¿cómo creamos el objeto correcto según el tipo de energía?** Para eso, Factory Method.”

5.2 Factory Method

Idea en una frase.

> "Una fábrica recibe un **tipo** y devuelve **la clase concreta correcta**, sin que el código de negocio use `new` por todas partes."

Problema y solución en el proyecto. La plataforma maneja varios **tipos/fuentes de energía** (solar, eólica, hidráulica) y varios **períodos de predicción** (diario, semanal). Si cada parte del sistema creara esos objetos a mano, añadir un tipo nuevo obligaría a editar muchos archivos. La fábrica centraliza la creación: basta con registrar la nueva clase en la fábrica.

Participantes y archivos (3 implementaciones).

Fábrica	Archivo	Productos
`EnergySourceFactory`	`backend-laravel/app/Patterns/Factories/EnergySourceFactory.php`	`SolarEnergy`, `WindEnergy`, `HydroEnergy`
`PredictorFactory`	`backend-laravel/app/Patterns/Factories/PredictorFactory.php`	`DailyPredictor`, `WeeklyPredictor`
`EnergyCardFactory`	`frontend/src/patterns/factory/EnergyCardFactory.tsx`	`SolarCard`, `WindCard`, `HydroCard`

Producto abstracto (backend): `abstract class EnergySource` con `getType()`, `getName()`, `getDescription()`, `getEfficiency()`, `getColor()` y `toArray()`.

Código clave (fábrica backend):

```
class EnergySourceFactory
{
    public const TYPES = ['solar', 'wind', 'hydro'];

    public function create(string $type): EnergySource
    {
        return match (strtolower($type)) {
            'solar' => new SolarEnergy(),
            'wind'  => new WindEnergy(),
            'hydro' => new HydroEnergy(),
            default => throw new \InvalidArgumentException("Tipo de energia no soportado:
{$type}"),
        };
    }

    public function createAll(): array
    {
        return array_map(fn (string $t) => $this->create($t), self::TYPES);
    }
}
```

Código clave (fábrica frontend):

```
class EnergyCardFactory {
  create(type: string): (props: { source: EnergySource }) => JSX.Element {
    switch (type) {
      case 'solar': return SolarCard;
      case 'wind':  return WindCard;
      case 'hydro': return HydroCard;
      default: throw new Error(`Tipo de energia no soportado: ${type}`);
    }
  }
}
export default new EnergyCardFactory();   // instancia única compartida
```

Dónde se ve en la app y endpoints.

- `GET /api/energy/source-catalog` devuelve el catálogo creado por la fábrica.
- El paso 2 del asistente de publicación (`CreateOffer`) muestra las tarjetas de energía que produce `EnergyCardFactory`.
- `MarketService` / `EnergyOfferBuilder` **validan el tipo** de energía con la fábrica al registrar una oferta: un tipo inválido produce HTTP 422.

Frase de cierre / transición.

> "La fábrica decide **qué** crear; ahora necesitamos que esos objetos compartan **cómo** se calcula un resultado. Eso es el Template Method."

5.3 Abstract Method / Template Method

Idea en una frase.

> "Una clase abstracta define el **esqueleto del algoritmo** y los pasos que cada subclase debe completar; la lógica común se escribe una sola vez."

Problema y solución en el proyecto. El subsistema de predicción estima producción, consumo, excedente/déficit, una acción (`publish` / `buy` / `none`) y una recomendación. Toda esa lógica es **igual** para una predicción diaria o semanal; solo cambian los **datos/estimaciones**. La clase abstracta `predict()` implementa el esqueleto y las subclases solo aportan los detalles.

Participantes y archivos (2 implementaciones).

Capa	Archivo	Clases
Backend	`app/Patterns/Predictors/AbstractPredictor.php` (+ `DailyPredictor.php`, `WeeklyPredictor.php`)	Clase base + 2 subclases
Frontend	`frontend/src/patterns/template/Predictor.ts` (+ `DailyPredictor.ts`, `WeeklyPredictor.ts`)	Clase base + 2 subclases

Cómo funciona.

1. La clase base declara métodos **abstractos**: `getPeriod()`, `getProductionEstimate()`, `getConsumptionEstimate()`, `getDescription()`, `getColor()`.

2. La clase base implementa el **método plantilla** `predict()`, que llama a esos métodos y calcula excedente/déficit, acción y recomendación usando el **contexto real del mercado** (`MarketService::getMarketContext()`).
3. Cada subclase solo define los valores (diario usa el dato directo; semanal lo escala ×6).

Código clave (backend):

```
abstract class AbstractPredictor
{
    abstract public function getPeriod(): string;
    abstract public function getProductionEstimate(): float;
    abstract public function getConsumptionEstimate(): float;

    final public function predict(): array   // ← método plantilla
    {
        $production  = $this->getProductionEstimate();
        $consumption = $this->getConsumptionEstimate();
        $surplus     = $production - $consumption;

        return [
            'period' => $this->getPeriod(),
            'surplusEstimate' => round(max(0, $surplus), 2),
            'deficitEstimate' => round(max(0, -$surplus), 2),
            'action' => $this->getAction($surplus),           // publish/buy/none
            'recommendation' => $this->buildRecommendation($surplus),
            'market' => $this->marketContext,
        ];
    }
}

class WeeklyPredictor extends AbstractPredictor {
    public function getProductionEstimate(): float {
        return $this->production > 0 ? $this->production * 6 : 21.5;   // solo cambia el dato
    }
}
```

Dónde se ve en la app y endpoints.

- `PredictionPanel` tiene dos botones: **“Calcular local”** ejecuta el patrón en TypeScript y **“Consultar backend”** lo ejecuta en PHP (`GET /api/energy/predictions`).
- La recomendación muestra cifras **reales** del mercado (precio promedio, ofertas activas).
- El panel permite **accionar** la predicción: si hay excedente, publica una oferta real; si hay déficit, compra energía real.

Frase de cierre / transición.

> “Ya sabemos predecir; ahora queremos **publicar ofertas construidas paso a paso**. Ese es el Builder.”

5.4 Builder

Idea en una frase.

> “Construye un objeto **complejo paso a paso**, separando el proceso de construcción de la representación final, y permite **variantes** con el mismo código.”

Problema y solución en el proyecto. Publicar una oferta implica varios campos (productor, tipo, cantidad, precio, descripción). En lugar de un gran constructor con muchos parámetros, un **builder** expone pasos encadenables y un **director** orquesta distintos “recetarios”: la oferta **personalizada** y la oferta **exprés** (que completa el precio por defecto desde `ConfigService`).

Participantes y archivos (2 implementaciones).

Capa	Archivos	Rol
Backend	`app/Patterns/Builders/EnergyOfferBuilder.php`, `OfferDirector.php`	Builder + Director
Frontend	`frontend/src/patterns/builder/EnergyOfferBuilder.ts`, `OfferDirector.ts`	Builder + Director

Cómo funciona.

- **Builder:** `reset()`, `setProducer()`, `setEnergyType()`, `setQuantity()`, `setPricePerKwh()`, `setDescription()`, `build()`. Valida cantidad y precio y delega el tipo en `EnergySourceFactory`.
- **Director:** `buildFromRequest($data)` (oferta completa) y `buildExpressOffer($nombre, $kwh, $tipo)` (precio por defecto de `ConfigService`).

Código clave (director backend):

```
class OfferDirector
{
    public function __construct(private EnergyOfferBuilder $builder) {}

    public function buildFromRequest(array $data): EnergyOffer
    {
        return $this->builder
            ->reset()
            ->setProducer($data['producerName'])
            ->setEnergyType($data['energyType'] ?? '')
            ->setQuantity((float) $data['totalKwh'])
            ->setPricePerKwh((float) $data['pricePerKwh'])
            ->setDescription($data['description'] ?? '')
            ->build();
    }

    public function buildExpressOffer(string $p, float $kwh, string $type): EnergyOffer
    {
        $default = (float) ConfigService::getInstance()->get('defaultPricePerKwh');
        return $this->builder->reset()->setProducer($p)->setEnergyType($type)
            ->setQuantity($kwh)->setPricePerKwh($default)->build();
    }
}
```

Dónde se ve en la app y endpoints.

- `CreateOffer` es un **asistente de 6 pasos** (productor, fuente, cantidad, precio, descripción, revisar) que al publicar usa el director.

- `POST /api/energy/offers` (oferta personalizada) y `POST /api/energy/offers/express` (oferta exprés con precio sugerido).
- `PredictionPanel` también usa el director para convertir un excedente estimado en oferta real.

Frase de cierre / transición.

> "Construir está bien... pero muchas veces queremos **copiar** una oferta parecida. Para eso, Prototype."

5.5 Prototype

Idea en una frase.

> "En vez de construir desde cero, un objeto **se copia a sí mismo** y entrega un clon independiente."

Problema y solución en el proyecto. Un productor suele publicar ofertas **similares** una y otra vez. Reconstruirlas campo a campo es tedioso. Con Prototype se **duplica una oferta existente** o se parte de una **plantilla** por tipo de energía (`PrototypeRegistry`), y se personaliza el clon.

Participantes y archivos (2 implementaciones).

Capa	Archivos	Rol
Backend	`Prototype.php` , `OfferPrototype.php` , `PrototypeRegistry.php`	Interfaz + prototipo + registro
Frontend	`Prototype.ts` , `OfferPrototype.ts` , `PrototypeRegistry.ts`	Interfaz + prototipo + registro

Cómo funciona (clave: clon independiente).

`OfferPrototype::copy()` genera un **nuevo objeto** con: `id = null` , `availableKwh = totalKwh` (stock reiniciado) y `createdAt = ''` . Modificar el clon **no afecta** al original. El `PrototypeRegistry` (Singleton) guarda **plantillas base por tipo** (sembradas con `EnergySourceFactory` + `ConfigService`) y devuelve una **copia**, nunca la plantilla almacenada.

Código clave (backend):

```
interface Prototype { public function copy(): Prototype; }

class OfferPrototype implements Prototype
{
    public function __construct(private EnergyOffer $offer) {}

    public function copy(): Prototype
    {
        $copy = clone $this->offer;          // copia profunda del modelo
        $copy->id = null;                    // el clon es un objeto nuevo
        $copy->availableKwh = $copy->totalKwh;
        $copy->createdAt = '';
        return new self($copy);
    }
}
```

Dónde se ve en la app y endpoints.

- `POST /api/energy/offers/{id}/clone` → duplica una oferta existente.
- `GET /api/energy/prototype-templates` → lista las plantillas del registro.
- `POST /api/energy/offers/clone-template` → crea una oferta clonando una plantilla.
- En `EnergyDashboard` el botón **"Duplicar oferta"** clona; `OfferPrototypePanel` muestra el clon **local (TS)** y la publicación del clon en el backend.

Frase de cierre / transición.

> "Del mercado pasamos al **mundo físico simulado**: dispositivos que hablan distinto. Para unificarlos, el Adapter."

5.6 Adapter

Idea en una frase.

> "Convierte la interfaz incompatible de una clase en **la interfaz que el cliente espera**."

Problema y solución en el proyecto. El subsistema IoT conecta dispositivos **heredados y heterogéneos** (medidores SCADA con registros de texto y inversores fotovoltaicos con payloads JSON). El resto del sistema no debe conocer esos formatos: los **adaptadores** los traducen a una interfaz común (`DeviceGateway`), con un modelo común (`DeviceTelemetry`).

Participantes y archivos (2 implementaciones).

Capa	Archivos	Rol
Backend	`Adapters/DeviceGateway.php` , `LegacyScadaAdapter.php` , `PvInverterAdapter.php` , `DeviceGatewayRegistry.php` , `DeviceMonitorService.php`	Interfaces + adaptadores + cliente
Frontend	`adapter/DeviceGateway.ts` , `InverterGatewayAdapter.ts` , `ScadaGatewayAdapter.ts` , `deviceGatewayRegistry.ts`	Mismo patrón en el navegador

Cómo funciona.

- **Target (lo que el cliente conoce):** `DeviceGateway` → `getProviderKey()`, `getDeviceId()`, `getDisplayName()`, `isOnline()`, `getSupportedMetrics()`, `readTelemetry()`.
- **Adaptees (lo viejo):** `LegacyScadaMeter` (registros crudos) y `PvInverterDevice` (JSON crudo).
- **Adapters:** traducen cada formato a `DeviceTelemetry`.
- **Cliente:** `DeviceMonitorService` solo conoce `DeviceGateway` → lista dispositivos, lee telemetría y calcula un resumen consolidado.

Dónde se ve en la app y endpoints.

Endpoint	Qué muestra
`GET /api/energy/legacy/meters`	Datos **crudos** del SCADA (el "antes")
`GET /api/energy/legacy/inverters`	Payload **crudo** del inversor
`GET /api/energy/devices`	Inventario **normalizado** (el "después")
`GET /api/energy/devices/{id}/telemetry`	Telemetría traducida
`GET /api/energy/devices/telemetry/summary`	Consolidado (total kW, kWh, por fabricante)

`DevicePanel` muestra lado a lado los datos crudos y los adaptados.

Frase de cierre / transición.

> "Con los datos ya unificados, hay que **facturarlos** en distintos perfiles y monedas. El Bridge separa esas dos dimensiones."

5.7 Bridge

Idea en una frase.

> "Separa una **abstracción** de su **implementación** para que ambas varíen independientemente, evitando una explosión de clases."

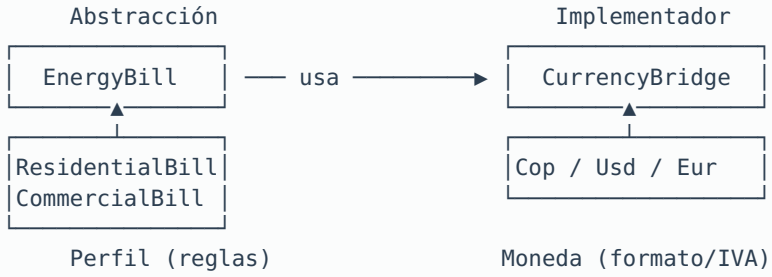
Problema y solución en el proyecto. La liquidación de energía depende de **dos ejes independientes**: el **perfil** del usuario (residencial/comercial) y la **moneda** (COP/USD/EUR). Sin Bridge tendríamos 2 × 3 = 6 clases (una por combinación) y crecería de forma multiplicativa. Con Bridge:

- **Abstracción:** `EnergyBill` (`ResidentialBill`, `CommercialBill`) → reglas del perfil.
- **Implementador:** `CurrencyBridge` (`CopCurrency`, `UsdCurrency`, `EurCurrency`) → formato, IVA, conversión y unidades mínimas.
- `SettlementService` conecta ambos ejes dinámicamente.

Participantes y archivos (2 implementaciones).

Capa	Archivos	Rol
Backend	`Bridges/EnergyBill.php` (abstracta), `ResidentialBill.php`, `CommercialBill.php`, `CurrencyBridge.php` (interfaz), `Cop/Usd/EurCurrency.php`, `SettlementService.php`	Abstracción + implementadores + cliente
Frontend	`bridge/EnergyBill.ts`, `ResidentialBill.ts`, `CommercialBill.ts`, `CurrencyBridge.ts`, `Cop/Usd/EurCurrency.ts`, `SettlementService.ts`	Mismo patrón

Cómo funciona.



`EnergyBill::build()` calcula el subtotal, aplica las reglas del perfil y **todo lo monetario** (IVA, formato, decimales) se lo pide al implementador de moneda.

Código clave:

```
abstract class EnergyBill {
    public function __construct(protected CurrencyBridge $currency) {}
    abstract protected function applyProfileRules(float $subtotal, array $data): array;
}

class CommercialBill extends EnergyBill {
    applyProfileRules($subtotal, $data): array {
        $flatFee = $this->currency->convertFromUsd(25.0);   // 97.500 COP | 25 USD | 23 EUR
        return [ /* recargo 2% + tarifa fija */ ];
    }
}
```

Dónde se ve en la app y endpoints.

- `GET /api/energy/settlements/matrix` → matriz perfiles × monedas (2 × 3 = 6 combinaciones).
- `POST /api/energy/settlements/quote` → emite la liquidación en la moneda y perfil pedidos.
- `SettlementPanel` deja cambiar perfil y moneda por separado y compara cálculo local vs backend.

Frase de cierre / transición.

> "La liquidación da un precio base; para **añadir extras** sin tocar ese base, el Decorator."

5.8 Decorator

Idea en una frase.

> "Añade **responsabilidades a un objeto dinámicamente**, envolviéndolo, en lugar de crear una clase por cada combinación."

Problema y solución en el proyecto. Una oferta puede llevar extras opcionales: **certificación verde (+5%)**, **seguro (+3%)** y **entrega prioritaria (+$8.000 COP)**. Con 3 extras hay $2^3 = 8$ combinaciones; crear una clase por combinación sería inviable. El Decorator permite **envolver** una oferta base con uno o varios decoradores, en cualquier orden y cantidad.

Participantes y archivos (2 implementaciones).

Capa	Archivos	Rol
Backend	`Decorators/OfferComponent.php` (interfaz), `BasicOffer.php`, `OfferDecorator.php`, `GreenCertificationDecorator.php`, `InsuranceDecorator.php`, `PriorityDeliveryDecorator.php`, `OfferPricingService.php`	Componente + decoradores + cliente
Frontend	`decorator/` (mismos nombres) + `OfferExtrasPanel.tsx`	Mismo patrón

Cómo funciona.

```
OfferComponent (interfaz)
      ▲                    ▲
      |                    | extiende
  BasicOffer          OfferDecorator (abstracto, envuelve otro OfferComponent)
                           ▲              ▲              ▲
                      Green(+5%)    Insurance(+3%)   Priority(+$8.000)
```

Cada decorador **delega** en el componente que envuelve y **añade** su ajuste a la descripción y al total. `OfferPricingService` es el cliente que aplica los extras solicitados.

Código clave:

```
$offer = new BasicOffer(20, 500);                    // 20 kWh × 500 = 10.000
$offer = new GreenCertificationDecorator($offer);    // +5%  → 10.500
$offer = new PriorityDeliveryDecorator($offer);      // +8.000 → 18.500
echo $offer->getTotal();                             // 18500
```

Dónde se ve en la app y endpoints.

- `GET /api/energy/extras` → catálogo de decoradores disponibles.
- `POST /api/energy/offers/quote-extras` → cotiza una oferta decorada.
- `OfferExtrasPanel`: marcas/desmarcas extras y el precio se recompone; se compara el total local (TS) con el del backend (PHP).

Frase de cierre / transición.

> "Con las ofertas decoradas, ya podemos **agruparlas en paquetes**: el Composite."

5.9 Composite

Idea en una frase.

> "Compón objetos en **estructuras de árbol** y trátalos igual (hoja o contenedor) para calcular totales de forma **recursiva**."

Problema y solución en el proyecto. Un **paquete de energía** puede contener ofertas individuales **u otros paquetes** (anidados). El cliente no debería preocuparse por si un nodo es hoja o grupo: ambos implementan `BundleComponent` y los totales de kWh y precio se **suman recursivamente** sobre el subárbol.

Participantes y archivos (2 implementaciones).

Capa	Archivos	Rol
Backend	`Composite/BundleComponent.php` (interfaz), `OfferLeaf.php` (hoja), `EnergyBundle.php` (compuesto), `BundleService.php`	Interfaz + hoja + compuesto + cliente
Frontend	`composite/` (mismos nombres) + `BundlePanel.tsx`	Mismo patrón

Cómo funciona.

```
BundleComponent (interfaz: getTotalKwh, getTotalPrice, countLeaves, getChildren)
      ▲                              ▲
  OfferLeaf                      EnergyBundle
  (1 oferta: suma la suya)       (N hijos: suma la de TODOS, recursivamente)
```

Código clave:

```
$solar = new EnergyBundle('Paquete Solar');
$solar->add(new OfferLeaf('Paneles techo', 15, 480, 'solar'))
      ->add(new OfferLeaf('Excedente patio', 5, 500, 'solar'));

$market = new EnergyBundle('Paquete del Mercado');
$market->add($solar)->add($renewable);

echo $market->getTotalPrice();   // se calcula solo, sin importar la profundidad
```

Dónde se ve en la app y endpoints.

- `GET /api/energy/bundles/sample` → árbol de ejemplo con totales recursivos.
- `POST /api/energy/bundles/evaluate` → evalúa un árbol JSON (cada nodo `type: "offer"` o `"bundle"`).
- `BundlePanel` dibuja el árbol, permite **añadir ofertas** al paquete raíz (los totales se recalculan) y **comparar** con el cálculo del backend.

Frase de cierre / transición.

> "Ya tenemos muchos subsistemas; para que el cliente no tenga que conocerlos todos, ponemos un Facade."

5.10 Facade

Idea en una frase.

> "Ofrece una **interfaz simplificada y de alto nivel** a un conjunto de subsistemas, para que el cliente no tenga que conocerlos ni coordinarlos."

Problema y solución en el proyecto. Para operaciones simples, el controlador y el frontend tenían que conocer **muchos** servicios: `MarketService`, `LoggerService`, `ConfigService`, `PredictorFactory`, `DeviceMonitorService`, `SettlementService`, `OfferPricingService`, `BundleService`. El Facade expone **5 operaciones de negocio** y coordina todo internamente:

Operación	Qué hace internamente
`getDashboard()`	Mercado + ofertas + IoT + configuración + actividad, en **una sola llamada**
`publishSurplus()`	Construye (Builder), registra, contabiliza y loguea
`buyEnergy()`	Compra en el mercado **y** emite la liquidación (Bridge)
`getSurplusAdvice()`	Predicciones (Abstract Method) + contexto real + recomendación
`getIoTNetworkStatus()`	Inventario (Adapter) + resumen consolidado

Participantes y archivos (2 implementaciones).

Capa	Archivo	Rol
Backend	`Facades/EnergyPlatformFacade.php`	Único punto de entrada del servidor
Frontend	`facade/PlatformFacade.ts`	Fachada del navegador que delega en `EnergyApiService`

Código clave (backend):

```
class EnergyPlatformFacade
{
    public function __construct()
    {
        $this->market = MarketProxy::getInstance();     // Proxy → MarketService (Singleton)
        $this->logger = LoggerService::getInstance();
        $this->predictors = new PredictorFactory();
        $this->devices = new DeviceMonitorService((new DeviceGatewayRegistry())->all());
        $this->settlements = new SettlementService();
    }

    public function getDashboard(): array
    {
        return [
            'market' => $this->market->getMarketContext(),
            'offers' => /* ofertas disponibles */,
            'iot'    => $this->devices->getSummary(),
            'config' => $this->config->getAll(),
            'activity' => $this->logger->getMetrics(),
        ];
    }
}
```

Dónde se ve en la app y endpoints.

- `GET /api/energy/platform/dashboard` — panel unificado en una sola petición.
- `POST /api/energy/platform/publish-surplus` — publica y contabiliza.
- `POST /api/energy/platform/buy` — compra + liquidación.
- `GET /api/energy/platform/advice` — asesoría (predicción + mercado).
- `GET /api/energy/platform/iot-status` — estado de la red IoT.
- `PlatformPanel` es el cliente de la fachada: una sola llamada trae todo el panel.

Frase de cierre / transición.

> "El Facade coordina datos, pero con muchas ofertas repetiríamos información por todas partes. Para **compartirla**, el Flyweight."

5.11 Flyweight

Idea en una frase.

> "Comparte el **estado intrínseco** (lo que se repite) entre muchísimos objetos, dejando fuera el **estado extrínseco** (lo que es único de cada uno), para ahorrar memoria."

Problema y solución en el proyecto. Todas las ofertas de tipo solar comparten el mismo nombre ("Energía Solar"), descripción, eficiencia (85%) y color (#f39c12). Si cada oferta guardara/creara esa información, duplicaríamos lo mismo miles de veces. Con Flyweight:

- **Estado intrínseco (compartido):** descriptor del tipo de energía (nombre, descripción, eficiencia, color; y en el frontend: emoji, imagen, colores).
- **Estado extrínseco (único):** productor, kWh, precio, fecha → viven en `EnergyOffer`.

El resultado: **3 instancias compartidas** reutilizadas por **todas** las ofertas.

Participantes y archivos (2 implementaciones).

Capa	Archivo	Rol
Backend	`app/Patterns/Flyweights/EnergySourceFlyweightFactory.php`	Pool de flyweights (una instancia por tipo)
Frontend	`frontend/src/patterns/flyweight/EnergyTypeFlyweightFactory.ts`	Descriptores visuales compartidos

Cómo funciona.

- La fábrica del flyweight mantiene un **pool** `tipo → instancia`.
- Se **siembra** una vez usando `EnergySourceFactory` (Factory Method) y luego **se reutiliza**.
- Cada uso se cuenta (`totalUses`), lo que permite calcular las **instancias ahorradas** (`savedInstances = totalUses − instanciasCompartidas`).
- En la lista de ofertas, cada oferta se **enriquece** con el descriptor compartido (`energySource`), en lugar de repetir esos datos.

Código clave (backend):

```
class EnergySourceFlyweightFactory
{
    private array $pool = [];   // tipo → EnergySource (compartido)

    private function seedPool(): void   // se crea UNA vez cada descriptor
    {
        foreach (EnergySourceFactory::TYPES as $type) {
            $this->pool[$type] = (new EnergySourceFactory())->create($type);
        }
    }

    public function get(string $type): EnergySource   // siempre el MISMO objeto
    {
        $key = strtolower(trim($type));
        $this->metrics['totalUses']++;
        return $this->pool[$key] ??= (new EnergySourceFactory())->create($key);
    }

    public function describe(string $type): array
    {
        return $this->get($type)->toArray();   // estado intrínseco compartido
    }
}
```

Código clave (frontend):

```
const descriptor = energyTypeFlyweightFactory.get(offer.energyType);
// → MISMO objeto { name: 'Energia Solar', image: '/images/solar-energy.svg',
//                  color: '#f39c12', emoji: '☀️', ... } para todas las ofertas solares
```

Dónde se ve en la app y endpoints.

- `GET /api/energy/flyweight-stats` → `createdInstances`, `totalUses`, `savedInstances`, `usesByType` y los descriptores compartidos.
- `GET /api/energy/offers` devuelve cada oferta con su `energySource` compartido.
- `EnergyDashboard` usa el flyweight para la imagen y la etiqueta del tipo, **sin funciones duplicadas** (antes había `switch` repetidos en 3 componentes).
- `PatternsPanel` muestra en vivo los descriptores compartidos y los usos.

Frase de cierre / transición.

> "Ya compartimos lo que se repite; ahora falta **controlar el acceso** al mercado y no repetir consultas costosas: el Proxy."

5.12 Proxy

Idea en una frase.

> "Es un **sustituto** con la misma interfaz que el objeto real, que **controla el acceso**: puede cachear, crear de forma diferida o proteger (autorizar/limitar)."

Problema y solución en el proyecto. Consultar el mercado en cada petición es costoso y, además, las compras necesitan reglas de seguridad. El **Proxy** envuelve al servicio real con tres facetas:

1. **Proxy virtual:** el objeto real (`MarketService`) se crea **solo cuando se necesita**.
2. **Proxy de caché:** guarda la lista de ofertas durante un **TTL** (10 s) y la **invalida** al publicar, clonar o comprar; así evita recorrer el mercado en cada lectura.
3. **Proxy de protección:** valida las compras contra `ConfigService` (**mínimo 1 kWh, máximo 100 kWh**) y **bloquea** las que no cumplen la política.

Participantes y archivos (2 implementaciones).

Capa	Archivo	Rol
Backend	`app/Patterns/Proxies/MarketProxy.php`	Sustituto de `MarketService` (virtual + caché + protección)
Frontend	`frontend/src/patterns/proxy/EnergyApiProxy.ts`	Sustituto de `EnergyApiService` (caché TTL + protección)

Cómo funciona (backend).

```
class MarketProxy
{
    private ?MarketService $real = null;    // Proxy virtual: se crea bajo demanda

    private function real(): MarketService
    {
        return $this->real ??= MarketService::getInstance();
    }

    public function getAvailableOffers(): array
    {
        if ($this->isCacheValid()) {          // Proxy de caché
            return $this->fromCache();        // no toca el sujeto real
        }
        $offers = array_values($this->real()->getAvailableOffers());
        $this->writeCache($offers);           // guarda con TTL
        return $offers;
    }

    public function purchaseOffer(int $id, int $kwh): array
    {
        if ($kwh < 1)   return $this->blocked('la compra mínima es 1 kWh');
        if ($kwh > 100) return $this->blocked('máximo 100 kWh por operación');

        $result = $this->real()->purchaseOffer($id, $kwh);   // Proxy de protección superado
        if ($result['success']) $this->invalidateCache();    // coherencia de la caché
        return $result;
    }
}
```

Cómo funciona (frontend). `EnergyApiProxy` envuelve a `EnergyApiService` (Singleton): cachea las ofertas 10 s y valida la compra (1–100 kWh) antes de llamar al backend; al comprar con éxito, invalida su caché.

Dónde se ve en la app y endpoints.

- `GET /api/energy/proxy-stats` → `ttlSeconds`, `valid`, `entries`, `expiresIn` y política.

- `EnergyDashboard` obtiene las ofertas a través del Proxy (con caché y por envío por `EnergyApiService`), y las compras pasan por la validación.
- `PatternsPanel` muestra la caché (válida/expirada, entradas, TTL) y permite **forzar la invalidación**.
- Demo rápida: pedir `/offers` dos veces → el Proxy reporta caché válida; intentar comprar **999 kWh** → el Proxy responde "no se pueden comprar más de 100 kWh por operación".

Frase de cierre.

> "Con esto cerramos el ciclo: compartimos información (Flyweight) y controlamos el acceso (Proxy), sobre una arquitectura donde cada patrón resuelve un problema concreto."

---

6. Demo en vivo paso a paso

> Orden recomendado para que un patrón "prepare" el siguiente. Ten abiertas dos pestañas: la **app** (frontend) y una terminal para `curl`.

Preparación (antes de exponer):

```
# Terminal 1 — backend
cd backend-laravel && php artisan serve --port=8000

# Terminal 2 — frontend
cd frontend && npm run dev        # http://localhost:5173
```

Recorrido sugerido (≈5 minutos):

Paso	Acción en pantalla	Qué señalar	Patrón
1	Sección **Mercado**, paso 2 del asistente	Tarjetas de energía (solar/eólica/hidráulica)	Factory Method
2	Publicar una oferta (6 pasos)	El asistente construye la oferta	Builder
3	Sección **Análisis** → “Calcular local” y “Consultar backend”	Misma predicción en TS y PHP	Abstract Method
4	Botón **Duplicar oferta** en una tarjeta	Clon independiente	Prototype
5	Sección **Plantillas**	Clonar una plantilla	Prototype
6	Sección **Dispositivos**	Datos crudos vs normalizados	Adapter
7	Sección **Calcular liquidación**	Cambiar perfil y moneda por separado	Bridge
8	Sección **Opciones de oferta**	Marcar extras y ver el total recomponerse	Decorator
9	Sección **Paquetes de energía**	Añadir oferta y ver totales recursivos	Composite
10	Sección **Plataforma**	Una sola llamada trae todo el panel	Facade
11	Sección **Flyweight y Proxy**	Descriptores compartidos + estado de caché	Flyweight + Proxy
12	Terminal: compra 999 kWh	El Proxy rechaza la operación	Proxy

Comandos `curl` de la demo (terminal):

```
B=http://localhost:8000/api/energy

# 1) Ofertas (Proxy + Flyweight: cada oferta trae su descriptor compartido)
curl $B/offers

# 2) Flyweight: instancias compartidas vs usos
curl $B/flyweight-stats

# 3) Proxy: caché válida con TTL
curl $B/proxy-stats
curl $B/offers > /dev/null && curl $B/offers > /dev/null
curl $B/proxy-stats                      # ahora "valid": true

# 4) Proxy de protección: compra bloqueada
curl -X POST $B/offers/1/purchase -H "Content-Type: application/json" -d '{"kwh":999}'

# 5) Facade: panel unificado
curl $B/platform/dashboard

# 6) Predicción (Abstract Method)
curl "$B/predictions?production=30&consumption=50"
```

Qué decir en cada paso (una línea): el proyecto está pensado para que **cada patrón sea visible en pantalla**, no solo en el código: las tarjetas (Factory), el asistente (Builder), los clones (Prototype), los dispositivos (Adapter), la factura (Bridge), los extras (Decorator), los paquetes (Composite), el panel unificado (Facade), el panel de patrones (Flyweight + Proxy).

7. Comparaciones que suelen preguntar

Comparación	Diferencia esencial	En este proyecto
Factory Method vs Builder	Factory: decide **qué** clase crear en un paso. Builder: construye **paso a paso** un objeto complejo.	`EnergySourceFactory` (tipo → clase) vs `EnergyOfferBuilder` + `OfferDirector` (oferta por pasos)
Builder vs Prototype	Builder construye desde cero; Prototype **copia** un objeto existente.	Oferta nueva vs `cloneOffer` /plantilla
Adapter vs Bridge	Adapter hace **compatibles** interfaces ya existentes; Bridge **separa** abstracción e implementación diseñadas juntas.	`LegacyScadaAdapter` vs `EnergyBill` + `CurrencyBridge`
Decorator vs Proxy	Decorator **añade** comportamiento (envuelve y agrega); Proxy **controla el acceso** (misma interfaz, sin añadir al resultado).	Extras de oferta vs `MarketProxy` (caché/ protección)
Decorator vs Composite	Decorator envuelve **uno** y añade; Composite agrupa **muchos** y suma recursivamente.	Extras vs paquetes de energía
Facade vs Proxy	Facade **simplifica** muchos subsistemas (interfaz nueva), Proxy es un sustituto **1 a 1** que controla el acceso.	`EnergyPlatformFacade` vs `MarketProxy`
Flyweight vs Singleton	Singleton: **una** instancia total. Flyweight: **varias** instancias (una por estado intrínseco) compartidas por muchos usuarios.	`MarketService` vs pool `tipo→EnergySource`
Flyweight vs Prototype	Prototype **crea copias** independientes; Flyweight **comparte** objetos sin copiar.	Clonar oferta vs descriptor compartido por tipo
Template Method vs Strategy	Template Method usa **herencia** (esqueleto fijo en la base); Strategy usa **composición** (algoritmo intercambiable).	Aquí se usó Template Method: `AbstractPredictor::predict()`
Prototype vs Builder (variantes)	Prototype clona variantes ya existentes; Builder genera variantes nuevas con pasos.	Plantillas vs oferta exprés

8. Preguntas frecuentes del jurado y respuestas modelo

1. **¿Por qué usaron Singleton y no una variable global?** Porque el Singleton **controla la creación**, serializa el acceso y mantiene el estado en una sola instancia tipada. Además permite cargar/guardar el estado (JSON) en el constructor privado, cosa que una variable global no hace.
2. **¿No es el Singleton un anti-patrón por el acoplamiento global?** Lo es si se abusa. Aquí se usa solo para **servicios de estado compartido** (mercado, logs, config, cliente HTTP). Es un prototipo académico; en producción se inyectarían dependencias (contenedor IoC de Laravel).
3. **¿Dónde está el polimorfismo del Factory Method?** En `EnergySource`: la clase abstracta declara el contrato y `SolarEnergy`, `WindEnergy`, `HydroEnergy` lo implementan. La fábrica devuelve el **tipo base**, y el cliente solo llama métodos de la interfaz.
4. **¿Por qué Abstract Method y no Strategy?** Porque el **esqueleto** de la predicción (calcular excedente, acción, recomendación) es común y **fijo**; solo cambian los datos. Eso es exactamente un **Template Method**.
5. **¿Qué pasa si añado un tipo de energía nuevo?** Se crea la clase (p. ej. `GeothermalEnergy`), se registra en `EnergySourceFactory::TYPES` y en el `switch` de `EnergyCardFactory`; el resto del sistema no cambia. Eso demuestra la extensibilidad.
6. **¿Cómo prueban que el Builder construye bien la oferta?** El asistente valida cada paso; el builder valida cantidad y precio y delega el tipo en la fábrica (422 si es inválido). El backend responde con la oferta construida y el panel la muestra.
7. **¿Cómo garantizan que el clon del Prototype es independiente?** `copy()` hace `clone` del modelo y **reinicia** `id`, `availableKwh` y `createdAt`. Modificar el clon no altera el original; el registry devuelve una copia, nunca la plantilla.
8. **¿Qué ventaja real dio el Adapter?** Unificar dispositivos con formatos distintos (texto SCADA vs JSON PV) en una sola interfaz y un solo modelo (`DeviceTelemetry`), de modo que `DeviceMonitorService` y los paneles no conocen los formatos originales.
9. **¿Por qué el Bridge y no herencia múltiple?** PHP no tiene herencia múltiple. El Bridge permite variar **perfil** y **moneda** por separado y evita 6+ clases; añadir una moneda no toca los perfiles.
10. **¿Cómo se combinan los decoradores?** Se envuelven unos sobre otros: cada uno delega en el componente envuelto y añade su ajuste. El resultado es la suma acumulada, sin importar el orden.
11. **¿Cómo se calculan los totales del Composite?** Recursivamente: `getTotalKwh()` / `getTotalPrice()` de un paquete suman las de sus hijos; una hoja devuelve lo suyo. Así el árbol puede anidarse a cualquier profundidad.
12. **¿El Facade hace "de todo" o delega? Delega y coordina**; no reimplementa. Cada operación compone los subsistemas (Builder, Bridge, Adapter, Abstract Method) y expone un resultado de alto nivel.

13. **¿Qué comparte exactamente el Flyweight?** El estado **intrínseco**: descriptor del tipo de energía (nombre, color, eficiencia, imagen). El estado **extrínseco** (productor, kWh, precio) es de cada oferta y no se comparte.
14. **¿Cómo demuestran el ahorro del Flyweight?** Con `/flyweight-stats`: pocas **instancias creadas** frente a muchos **usos**, y el campo `savedInstances` (instancias que no hubo que crear).
15. **¿Qué tres tipos de Proxy implementaron?** Virtual (creación diferida de `MarketService`), de caché (ofertas con TTL e invalidación) y de protección (límites de compra). Los tres con la **misma interfaz**.
16. **¿Qué pasa con la caché cuando alguien compra o publica?** El Proxy la **invalida** en cada mutación (publicar, clonar, comprar). Así se evita servir ofertas desactualizadas.
17. **¿Por qué el mismo patrón está en backend y frontend?** Para demostrar el patrón en dos lenguajes y en dos responsabilidades: el backend protege/cachea el **servidor**, el frontend la **red**. Los paneles comparan ambos cálculos (por ejemplo, Decorator y Bridge).
18. **¿Dónde persisten los datos?** En archivos JSON dentro de `storage/app/`: `market_data.json`, `logger_data.json`, `flyweight_data.json`, `market_proxy_cache.json`. Es una base de datos en memoria persistida, como pide el alcance del proyecto.
19. **¿Qué patrón resuelve el objetivo de "clasificar los tipos de energía"?** Factory Method (`EnergySourceFactory`) al crear/validar tipos, y Flyweight al **compartir** los descriptores de cada tipo.
20. **¿Qué mejorarían para producción?** Inyección de dependencias, base de datos real, caché distribuida (Redis), autenticación y autorización reales en el Proxy de protección, y pruebas automatizadas.

---

9. Mapa de la rúbrica y los objetivos del proyecto

Rúbrica que sigue el README: cada patrón se documenta con (a) "En qué consiste dentro del proyecto", (b) **diagrama UML**, (c) **código clave/completo**, (d) **integración con el resto del sistema** (endpoints, componentes) y (e) se añade al **índice de patrones**. Todos los patrones de esta guía cumplen esos cinco puntos en el README.

Cumplimiento de los objetivos específicos:

Objetivo específico	Patrones que lo evidencian
Registrar y gestionar producción/consumo	Singleton (estado), Abstract Method (predicción)
Gestionar excedentes disponibles	Builder, Prototype, Composite, Flyweight
Clasificar tipos de energía	Factory Method, Flyweight
Realizar compras y ventas	Proxy (protección/caché), Facade (compra + liquidación)
Registrar y consultar historial	Singleton (`LoggerService`, `MarketService`), Facade
Predicción de producción/consumo	Abstract Method / Template Method
Evaluar con pruebas funcionales	Endpoints + paneles que comparan local vs backend

Cobertura por subsistema:

Subsistema	Patrones principales
Compra y venta de excedentes	Singleton, Factory, Builder, Prototype, Facade, Flyweight, Proxy
Subastas	(base del mercado) Prototype/Composite como soporte
Dispositivos IoT	Adapter
Predicción	Abstract Method

---

10. Endpoints y comandos útiles

Ejecutar el proyecto:

```
# Backend
cd backend-laravel
composer install
php artisan key:generate
php artisan serve --port=8000

# Frontend
cd frontend
npm install
npm run dev            # http://localhost:5173
```

Endpoints principales por patrón:

Método	Ruta	Patrón	Descripción
GET	`/api/energy/source-catalog`	Factory Method	Catálogo de fuentes
GET	`/api/energy/predictions`	Abstract Method	Predicciones (acepta `?production=&consumption=`)
POST	`/api/energy/offers`	Builder	Crear oferta personalizada
POST	`/api/energy/offers/express`	Builder	Oferta exprés (precio por defecto)
POST	`/api/energy/offers/{id}/clone`	Prototype	Duplicar oferta
GET	`/api/energy/prototype-templates`	Prototype	Ver plantillas
POST	`/api/energy/offers/clone-template`	Prototype	Crear desde plantilla
GET	`/api/energy/legacy/meters`	Adapter	Datos crudos SCADA
GET	`/api/energy/legacy/inverters`	Adapter	Payload crudo inversor
GET	`/api/energy/devices`	Adapter	Inventario normalizado
GET	`/api/energy/devices/{id}/telemetry`	Adapter	Telemetría traducida
GET	`/api/energy/devices/telemetry/summary`	Adapter	Resumen IoT
GET	`/api/energy/settlements/matrix`	Bridge	Perfiles × monedas
POST	`/api/energy/settlements/quote`	Bridge	Emitir liquidación
GET	`/api/energy/extras`	Decorator	Catálogo de extras
POST	`/api/energy/offers/quote-extras`	Decorator	Cotizar con extras
GET	`/api/energy/bundles/sample`	Composite	Árbol de ejemplo
POST	`/api/energy/bundles/evaluate`	Composite	Evaluar árbol
GET	`/api/energy/platform/dashboard`	Facade	Panel unificado
POST	`/api/energy/platform/publish-surplus`	Facade	Publicar excedente
POST	`/api/energy/platform/buy`	Facade	Comprar + liquidar
GET	`/api/energy/platform/advice`	Facade	Asesoría
GET	`/api/energy/platform/iot-status`	Facade	Estado IoT
GET	`/api/energy/flyweight-stats`	Flyweight	Métricas del pool
GET	`/api/energy/proxy-stats`	Proxy	Estado de caché/políticas

11. Glosario rápido

- **Estado intrínseco:** datos que se repiten entre muchos objetos (Flyweight).
- **Estado extrínseco:** datos únicos de cada objeto, que no se comparten.
- **Cache hit / cache miss:** la petición se sirve desde la caché / hay que ir al origen.
- **TTL (Time To Live):** tiempo durante el cual una caché se considera válida.
- **Proxy virtual:** retrasa la creación del objeto real hasta que se necesita.
- **Proxy de protección:** verifica permisos/reglas antes de permitir la operación.
- **Adaptee:** la clase "vieja" que no encaja con la interfaz esperada (Adapter).
- **Abstracción / Implementador:** los dos ejes que separa el Bridge.
- **Cliente:** quien consume un patrón (controlador, componente React, servicio).
- **Componente / Hoja / Compuesto:** los tres roles del Composite.
- **Director / Builder:** quien orquesta los pasos y quien los ejecuta (Builder).

---

12. Checklist final antes de exponer

Técnico

- [ ] Backend levantado (`php artisan serve`) y probado `/api/energy/offers`.
- [ ] Frontend levantado (`npm run dev`) y navegado por las 10 secciones.
- [ ] Endpoints nuevos responden: `/flyweight-stats`, `/proxy-stats`.
- [ ] Datos de demostración cargados (al menos 3–5 ofertas de distintos tipos).

De contenido

- [ ] Puedo decir la **idea en una frase** de los 12 patrones sin leer.
- [ ] Sé **por qué** cada patrón está en el proyecto (problema que resuelve).
- [ ] Puedo señalar **el archivo** de cada patrón y leer un fragmento clave.
- [ ] Tengo preparadas las **comparaciones** (sección 7).
- [ ] Tengo preparadas las **FRASES de transición** entre patrones.

De la demo

- [ ] Ensayé el recorrido de la sección 6 en orden.
- [ ] Sé qué decir al mostrar la compra bloqueada por el Proxy (999 kWh).
- [ ] Sé mostrar la caché pasando de "no válida" a "válida" con dos llamadas a `/offers`.

Cierre

- [ ] Cierro recordando que hay **12 patrones × 2 capas**, documentados en el README con UML, código e integración, y que cada patrón resuelve un problema **real** del proyecto.

Fin de la guía. ¡Éxito en la exposición!